SQL Summary — What We Learned

Quick revision sheet for the SQL topics we covered.

1. SELECT + FROM

SELECT chooses the columns you want to display.
FROM tells SQL which table to get the data from.

```
SELECT name, age
FROM Students;
```

2. WHERE

WHERE filters rows.

```
SELECT name
FROM Students
WHERE age > 18;
```

3. Comparison Operators

= equal to
<> not equal to
> greater than
< less than
>= greater than or equal to
<= less than or equal to

4. AND / OR

AND means all conditions must be true.
OR means at least one condition must be true.

```
SELECT name
FROM Students
WHERE age > 18 AND course = 'Engineering';
```

5. ORDER BY

ASC = smallest to biggest / A to Z
DESC = biggest to smallest / Z to A

```
SELECT name, age
FROM Students
ORDER BY age DESC;
```

6. Aggregate Functions

COUNT() = counts rows/values
AVG() = average
SUM() = adds values together
MIN() = smallest value
MAX() = largest value

Remember: SUM(quantity) gives total quantity; COUNT(quantity) counts quantity entries.

7. LIKE

LIKE is used for text patterns. % means any number of characters.

```
WHERE name LIKE 'M%'
WHERE name LIKE '%a'
WHERE name LIKE '%a%'

Starts with M / ends with a / contains a
```

8. GROUP BY

GROUP BY puts rows into groups so aggregate functions can be calculated for each group.

```
SELECT course, COUNT(*)
FROM Students
GROUP BY course;
```

9. HAVING

HAVING filters groups after GROUP BY. WHERE filters rows; HAVING filters grouped results.

```
SELECT course, AVG(age)
FROM Students
GROUP BY course
HAVING AVG(age) > 19;
```

10. INNER JOIN

INNER JOIN combines two tables when a matching value exists in both tables. ON tells SQL how the tables connect.

```
SELECT ...
FROM Students
INNER JOIN Courses
ON Students.course_id = Courses.course_id;
```

11. AS — Column Headings

AS gives a result column a different heading without changing the actual database column.

```
SELECT name AS 'StudentName'
FROM Students;
```

12. Report Query We Built

Revenue = price × quantity
Total quantity = SUM(quantity)
Total revenue = SUM(price * quantity)

```
SELECT sale_date AS 'SalesDate',
       product_name AS 'ProductName',
       SUM(quantity) AS 'TotalQuantity',
       SUM(price * quantity) AS 'TotalRevenue'
FROM Sales
INNER JOIN Products
ON Sales.product_id = Products.product_id
GROUP BY sale_date, product_name;
```

13. Big Rule

If you SELECT normal columns alongside an aggregate such as SUM(), COUNT(), or AVG(), the normal columns generally need to appear in GROUP BY.

```
SELECT sale_date, product_name, SUM(quantity)
FROM Sales ...
GROUP BY sale_date, product_name;
```

14. What We're NOT Covering

We deliberately removed DELETE, UPDATE, and other JOIN types such as LEFT JOIN and RIGHT JOIN. Focus: basics, aggregate functions, LIKE, GROUP BY, HAVING, and INNER JOIN.

Quick Revision Checklist

SELECT → what columns?
FROM → which table?
WHERE → which rows?
ORDER BY → how should results be sorted?
SUM / COUNT / AVG / MIN / MAX → what calculation?
LIKE → what text pattern?
GROUP BY → what groups?
HAVING → which groups should remain?
INNER JOIN → how do the tables connect?
AS → what heading should the result column have?